

推荐系统中的生成式检索

——《Recommender Systems with Generative Retrieval》逐段中文翻译

原文作者：Shashank Rajput, Nikhil Mehta, Anima Singh, 等

中文整理：ChatGPT

目录

题目与摘要	2
1 引言	2
2 相关工作	3
3 本文方法	4
3.1 语义 ID 的生成	5
3.2 基于语义 ID 的生成式检索	7
4 实验	8
4.1 序列推荐性能	9
4.2 物品表示分析	9
4.3 新能力	11
4.4 消融实验	12
4.5 无效 ID	13
5 结论	13
A 附录 A：相关工作的补充	14
B 附录 B：基线方法简介	14
C 附录 C：数据集统计	15

目录	2
D 附录 D: 对 P5 数据预处理的修改	15
E 附录 E: 进一步讨论	15
参考文献 (原文保留)	17

说明

本文依据用户提供的论文《Recommender Systems with Generative Retrieval》进行逐段中文翻译与整理。为便于阅读，本文保留原论文的章节结构、公式、表格数值与附录层次；图像采用原论文中的部分原始插图，并将图注翻译为中文。参考文献条目主要保留原始英文书目信息，以避免对文献信息造成二次歧义。

题目与摘要

题目：推荐系统中的生成式检索

摘要：现代推荐系统通常通过如下方式完成大规模召回：首先将查询与候选物品嵌入到同一个统一空间中，然后在该空间中使用近似最近邻搜索，根据查询嵌入选出得分最高的候选项。本文提出了一种新的生成式检索方法，其中检索模型以自回归方式解码目标候选项的标识符。为此，我们为每个物品构造具有语义意义的码字元组，并将其作为该物品的语义 ID (Semantic ID)。给定用户会话中各物品的语义 ID 后，我们训练一个基于 Transformer 的序列到序列模型，去预测用户下一次将要交互物品的语义 ID。实验表明，基于该范式训练得到的推荐系统在多个数据集上显著优于当前最先进模型。此外，我们还表明，将语义 ID 引入序列到序列模型能够增强其泛化能力，这一点在那些缺乏历史交互记录物品上的检索性能提升中得到了体现。

1 引言

推荐系统帮助用户发现自己感兴趣的内容，并且已经广泛存在于多种推荐场景中，例如视频、应用、商品以及音乐推荐。现代推荐系统通常采用“召回—排序”的策略：首先在召回阶段选出一组可行候选项，然后再由排序模型对这些候选项进行重排。由于排序模型只能处理它收到的候选项，因此人们希望召回阶段能够尽可能输出高相关性的候选项。

构建召回模型已经形成了一套标准且成熟的方法。矩阵分解学习查询和候选项在同一空间中的嵌入表示。为了更好地捕捉数据中的非线性，近年来双塔结构变得十分流行：一座塔用于编码查询，另一座塔用于编码候选项，并通过内积把两者嵌入到同一个空间中。在推理时，通常先利用候选塔为全部物品建立索引；对于给定查询，再利用查询塔得到查询嵌入，并使用近似最近邻算法完成召回。近年来，双塔结构也被扩展到了序列推荐任务中，以显式考虑用户—物品交互的顺序信息。

本文提出一种用于序列推荐的全新生成式检索范式。不同于传统的查询—候选项匹配方式，我们的方法采用一个端到端生成模型，直接预测候选物品的 ID。我们的核心思想是将 Transformer 的参数记忆直接用作推荐系统中的端到端检索索引，这与 Tay 等人在文档检索中利用 Transformer 记忆进行检索的思路相呼应。我们将该方法称为 **TIGER** (Transformer Index for GEnerative Recommenders)。

TIGER 的关键特征在于一种新颖的物品语义表示方式，称为“语义 ID”。语义 ID 是由物品内容信息导出的一个离散 token 序列。具体而言，给定物品的文本特征，我们使用预训练文本编码器（例如 SentenceT5）生成稠密内容嵌入，然后再对该嵌入施加量化过程，从而形成一个有序的 token / 码字集合，这个集合即该物品的语义 ID。最终，这些语义 ID 被用来训练 Transformer 模型完成序列推荐任务。

将物品表示为语义 token 序列具有多方面优势。首先，在具有语义意义的数据上训练 Transformer 记忆，可以使相似物品之间共享知识，因此不再需要此前推荐模型中常见的原子式、随机式物品 ID。其次，采用语义 token 表示后，模型更不容易受到推荐系统中固有反馈回路的影响，从而能够更好地泛化到新加入语料库的物品。再次，使用 token 序列表示物品，也有助于缓解超大规模物品库带来的表示挑战，因为一串 token 所能表示的物品数量等于各位置 token 基数的乘积。现实中物品规模常常达到十亿级，为每个物品单独学习一个嵌入会造成较高内存开销。虽然可以采用随机哈希等技术压缩表示空间，但本文表明，利用具有语义意义的 token 来表示物品，是一种更有吸引力的替代方案。

本文的主要贡献如下：

1. 提出 TIGER，这是一种新的基于生成式检索的推荐框架。该框架为每个物品分配语义 ID，并训练检索模型来预测用户可能交互物品的语义 ID。
2. 在多个数据集上，以 Recall 和 NDCG 为指标，TIGER 均优于现有最先进推荐系统。
3. 我们发现，这种生成式检索范式还为序列推荐系统带来了两项额外能力：其一，能够更好地推荐新物品和低频物品，从而改善冷启动推荐；其二，能够通过可调参数生成更具多样性的推荐结果。

论文结构。第 2 节简要回顾推荐系统、生成式检索以及本文所采用的语义 ID 生成技术；第 3 节介绍本文提出的框架，并说明用于生成语义 ID 的多种技术；第 4 节给出实验结果；第 5 节给出结论。

2 相关工作

序列推荐

深度序列模型在推荐系统中的应用已经发展为一个内容丰富的研究方向。GRU4REC 是最早将基于 GRU 的循环神经网络用于序列推荐的方法。Li 等人提出 NARM，在 GRU 层基础上引入注意力机制，用于跟踪用户的长期意图。Zhang 等人提出 AttRec，利用自注意力建模当前会话中的用户意图，并通过度量学习建模用户—物品亲和性以实现个性化。与此同时，Kang 等人提出 SASRec，使用类似 decoder-only Transformer 的自注意力结构。受到掩码语言建模成功经验的启发，BERT4Rec 与 Transformers4Rec 采用带有掩码策略的 Transformer 模型来解决序列推荐任务。S³-Rec 则进一步超越简单掩码，通过四个自监督任务进行预训练，以改善数据表示。上述方法通常都为每个物品学习一个高维嵌入，并在最

大内积搜索空间中通过 ANN 预测下一个物品。与之不同，本文提出的 TIGER 通过生成式检索，直接预测下一个物品的语义 ID。

P5 将预训练大型语言模型微调用于多任务推荐系统。P5 依赖 LLM 自带分词器 (SentencePiece) 来对随机分配的物品 ID 进行 token 化；而本文使用的是基于物品内容信息学习得到的语义 ID 表示。实验中我们将展示：基于语义 ID 的推荐系统，相较于使用随机编码的方案，能够取得更好的结果。

语义 ID

Hou 等人提出 VQ-Rec，利用内容信息生成“代码”（可视为语义 ID 的类似物）来表示物品。不过，他们的重点在于构建可迁移推荐系统，并未以生成方式使用这些代码进行检索。尽管他们同样采用了乘积量化来生成代码，本文则使用 RQ-VAE 生成语义 ID，这使得物品表示天然具有层次结构。与我们同期的一项工作表明，在大规模推荐排序模型中，用层次化语义 ID 替换原始 item ID，可以提升模型的泛化能力。

生成式检索

虽然过去已经有人研究过可学习索引结构，但“生成式检索”是近年来在文档检索中迅速发展起来的一类新方法，其目标是从数据库中返回与查询相关的一组文档。相关代表性工作包括 GENRE、DSI、NCI 与 CGR 等。据我们所知，本文是第一个将生成式检索与语义 ID 表示结合，用于推荐系统的工作。

3 本文方法

本文框架由两个阶段构成：

1. **基于内容特征生成语义 ID**。该阶段先把物品内容特征编码为嵌入向量，再将嵌入量化为一组语义码字元组，所得码字元组即该物品的语义 ID。
2. **在语义 ID 上训练生成式推荐系统**。该阶段使用物品语义 ID 序列训练 Transformer 模型，使其完成序列推荐任务。

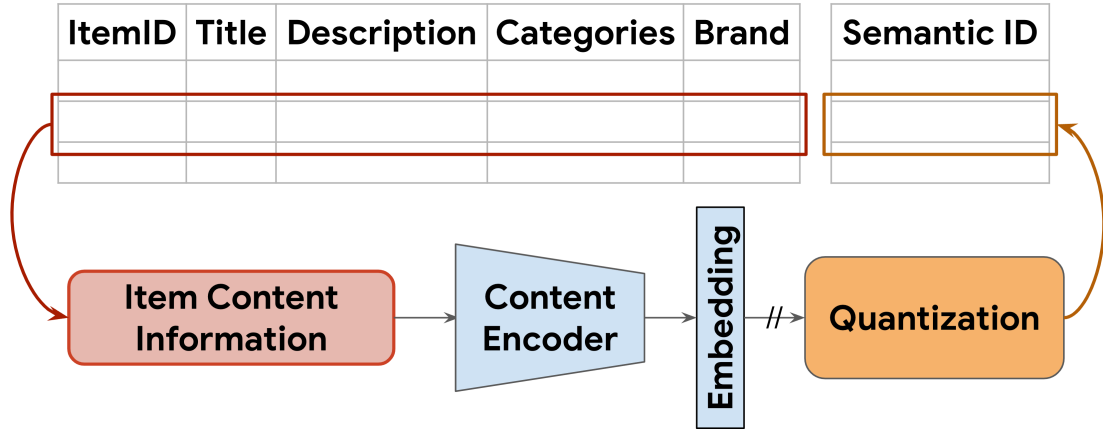


图 1: TIGER 中建模方法概览图的一部分：利用内容嵌入量化为物品生成语义 ID。

3.1 语义 ID 的生成

本节介绍推荐语料库中各物品语义 ID 的生成过程。我们假设每个物品都带有能够反映其语义信息的内容特征，例如标题、描述或图像；同时也假设我们拥有一个预训练内容编码器，可将其转换为语义嵌入 $x \in \mathbb{R}^d$ 。例如，可以使用 Sentence-T5 或 BERT 这类通用预训练文本编码器，将物品文本特征转换为语义嵌入。随后再对这些语义嵌入进行量化，从而为每个物品生成语义 ID。

我们将语义 ID 定义为一个长度为 m 的码字元组。元组中的每个码字都来自不同的码本，因此语义 ID 所能唯一表示的物品总数，等于各码本大小的乘积。不同的语义 ID 生成技术会赋予这些 ID 不同的语义性质，但至少应满足一个基本要求：**相似物品应当拥有部分重叠的语义 ID**。换言之，若两个物品内容特征相似，或其语义嵌入彼此接近，那么它们的语义 ID 也应更相近。比如，语义 ID 为 (10, 21, 35) 的物品，应比语义 ID 为 (10, 23, 32) 的物品，更接近语义 ID 为 (10, 21, 40) 的物品。下面我们介绍本文所采用的量化方案。

用于语义 ID 的 RQ-VAE。 残差量化变分自编码器 (Residual-Quantized Variational AutoEncoder, RQ-VAE) 是一种多层向量量化器，它通过对残差逐层量化，生成一个码字元组，也就是本文的语义 ID。自编码器在训练过程中同时更新量化码本以及 DNN 编码器、解码器参数。该过程可以视为从粗到细地逐步逼近输入表示。

RQ-VAE 首先通过编码器 E 将输入 x 编码为潜在表示 $z := E(x)$ 。在第零层 $d = 0$ ，初始残差定义为 $r_0 := z$ 。在每一层 d ，都存在一个码本 $C_d = \{e_k\}_{k=1}^K$ ，其中 K 为码本大小。然后，将当前残差映射到该层码本中距离最近的向量，其索引即该层的码字。例如在 $d = 0$ 时，找到距离 r_0 最近的嵌入 e_{c_0} ，则 c_0 就是第一个码字。接着定义下一层残差为 $r_1 := r_0 - e_{c_0}$ ，并在第二层码本中为 r_1 找到最近向量，从而得到第二个码字。如此递归进行 m 次，最终得到长度为 m 的码字元组，作为该物品的语义 ID。

这种递归式残差量化实际上是在以由粗到细的粒度逼近输入。本文为每一层都使用一个独立的、大小为 K 的码本，而不是使用一个大小为 mK 的统一码本。原因在于：随着

层数增加，残差的范数通常会逐渐减小，因此让不同层使用不同粒度的码本更为合理。

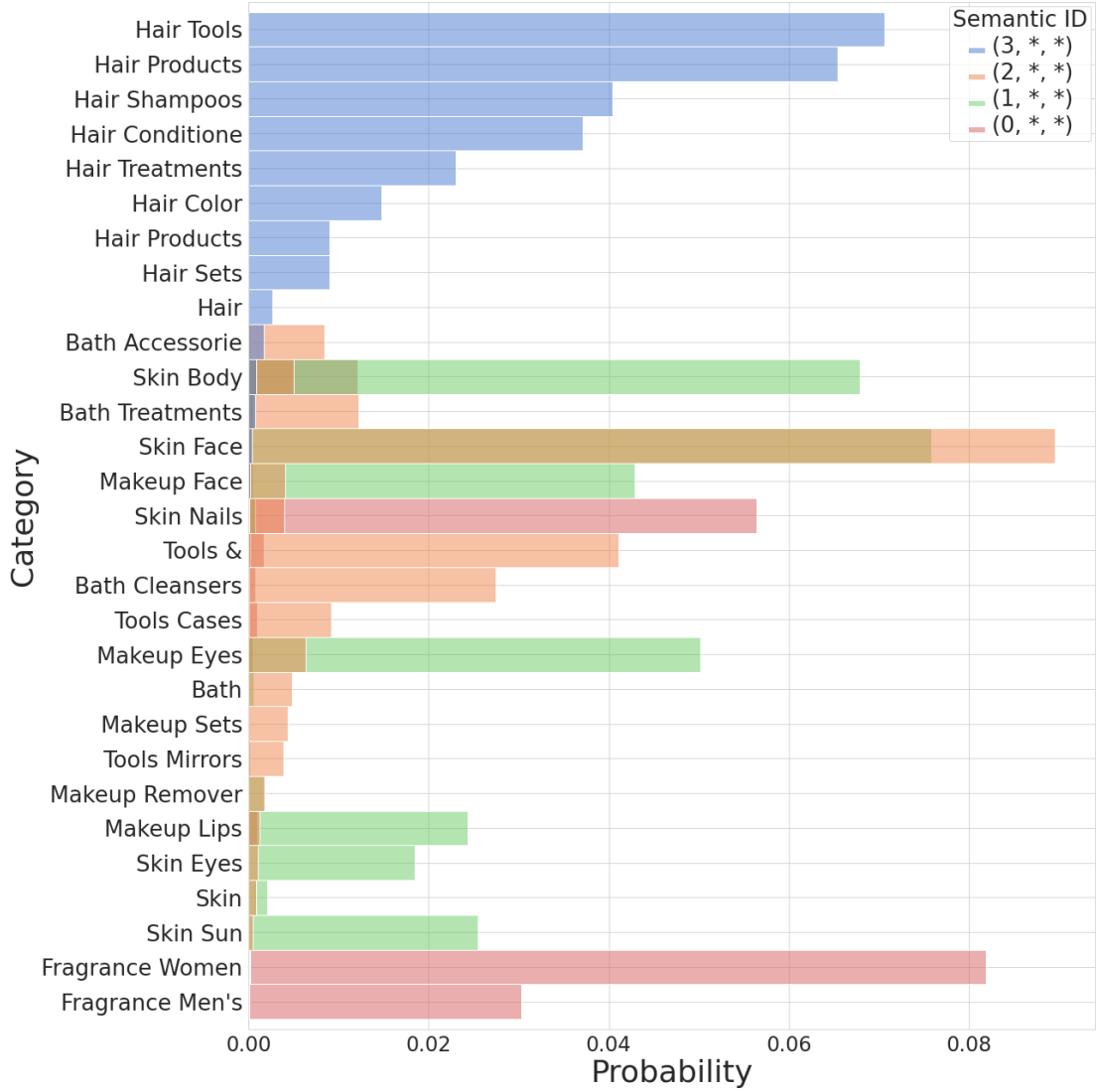


图 2: RQ-VAE 语义 ID 的定性分析示意图之一：第一层语义码能够对应较粗粒度的类别结构。

当得到语义 ID $(c_0, \dots, c_{m-1})$ 后，可以计算潜在表示 z 的量化近似表示：

$$\hat{z} = \sum_{d=0}^{m-1} e_{c_d}.$$

然后将 $\hat{z}$ 输入解码器，重建原始输入 x 。RQ-VAE 的损失函数写为

$$L(x) := L_{\text{recon}} + L_{\text{rqvae}},$$

其中

$$L_{\text{recon}} := \|x - \hat{x}\|^2,$$

而

$$L_{\text{rqvae}} := \sum_{d=0}^{m-1} \|\text{sg}[r_d] - e_{cd}\|^2 + \beta \|r_d - \text{sg}[e_{cd}]\|^2.$$

这里 $\hat{x}$ 为解码器输出, sg 表示 stop-gradient 操作。该损失会联合训练编码器、解码器以及码本。

按照原论文中的做法, 为了避免 RQ-VAE 的码本坍塌问题, 也就是大量输入只映射到少数几个码本向量, 作者采用基于 k-means 聚类的初始化策略: 在第一批训练数据上执行 k-means, 并将聚类中心作为码本的初始值。

其他量化替代方案。 一种简单替代方式是使用局部敏感哈希 (LSH) 生成语义 ID。作者在实验消融中发现, RQ-VAE 的效果优于 LSH。另一种选择是使用分层 k-means 聚类, 但这种方法会损失不同簇之间的语义连贯性。作者还尝试了 VQ-VAE。虽然在候选生成效果上它与 RQ-VAE 相近, 但它缺少 RQ-VAE 所带来的层次化 ID 特性, 而后者正是冷启动与多样性等新能力的重要基础。

碰撞处理。 由于语义嵌入分布、码本大小与码字长度的选择等因素, 可能会出现语义碰撞, 即多个物品被映射到相同语义 ID。为解决这一问题, 作者在有序语义码的末尾再附加一个额外 token, 以保证唯一性。比如, 如果两个物品都共享语义 ID (12, 24, 52), 则可分别表示为 (12, 24, 52, 0) 与 (12, 24, 52, 1)。为检测碰撞, 系统维护一个从语义 ID 到物品的查找表。值得注意的是, 碰撞检测与修复只需在 RQ-VAE 训练完成后执行一次; 此外, 由于语义 ID 本质上只是整数元组, 这个查找表在存储上也比高维嵌入更为高效。

3.2 基于语义 ID 的生成式检索

作者首先将每个用户交互过的物品按照时间顺序排列, 从而为每个用户构建物品序列。给定形如 $(\text{item}_1, \dots, \text{item}_n)$ 的序列, 推荐系统的任务是预测下一件物品 item_{n+1} 。本文提出以生成方式直接预测下一物品的语义 ID。

形式化地说, 设 item_i 的长度为 m 的语义 ID 为 $(c_{i,0}, \dots, c_{i,m-1})$ 。那么原始物品序列就被转换为如下 token 序列:

$$(c_{1,0}, \dots, c_{1,m-1}, c_{2,0}, \dots, c_{2,m-1}, \dots, c_{n,0}, \dots, c_{n,m-1}).$$

然后, 序列到序列模型被训练去预测 item_{n+1} 的语义 ID, 即 $(c_{n+1,0}, \dots, c_{n+1,m-1})$ 。由于该框架本质上是生成式的, 解码器也有可能生成一个在推荐语料库中不存在的语义 ID。不过, 原论文附录中的结果表明, 这种情况发生的概率较低; 作者也进一步讨论了此类情况的处理方式。

4 实验

数据集

作者在三个公开真实数据集上评估所提框架，这些数据集均来自 Amazon Product Reviews 数据集，覆盖 1996 年 5 月至 2014 年 7 月期间的用户评论及物品元数据。具体采用了其中三个类别用于序列推荐任务：“Beauty”“Sports and Outdoors”与“Toys and Games”。更多数据统计与预处理细节见附录。

评价指标

实验采用 Top- k Recall (Recall@K) 与归一化折损累计增益 (NDCG@K) 作为评价指标，其中 $K \in \{5, 10\}$ 。

RQ-VAE 实现细节

如第 3.1 节所述，RQ-VAE 用于对物品语义嵌入进行量化。作者使用预训练 Sentence-T5 模型生成数据集中每个物品的语义嵌入。具体地，将物品的标题、价格、品牌、类别等内容字段拼接成一句文本，再输入 Sentence-T5，得到 768 维语义嵌入。

RQ-VAE 由三部分构成：一个 DNN 编码器，用于把输入语义嵌入编码为潜在表示；一个残差量化器，用于输出量化表示；以及一个 DNN 解码器，用于把量化表示还原回语义嵌入空间。编码器包含三个中间层，大小分别为 512、256 与 128，激活函数为 ReLU，最终潜在表示维度为 32。为量化这一表示，作者采用三层残差量化；每层维护一个大小为 256 的码本，码本中每个向量维度均为 32。总损失中的超参数取 $\beta = 0.25$ 。为了保证码本使用率较高（不低于 80%），RQ-VAE 共训练 20k 个 epoch。优化器使用 Adagrad，学习率为 0.4，batch size 为 1024。训练完成后，作者使用学习到的编码器与量化模块，为每个物品生成一个长度为 3 的语义 ID。为了避免多个物品映射到同一语义 ID，对于前三个码字相同的物品，再补加一个唯一的第 4 个码字；即使不存在碰撞，也将第 4 个码字赋为 0。最终，每个物品都获得长度为 4 的唯一语义 ID。

序列到序列模型实现细节

作者使用开源 T5X 框架实现 Transformer 编码器—解码器结构。为使模型能够处理序列推荐任务，词表中包含了各语义码字对应的 token。具体而言，物品语义 ID 共由 $256 \times 4 = 1024$ 个 token 表示。此外，作者还向词表中加入用户特定 token。为了限制词表规模，只保留 2000 个用户 ID token，并使用 Hashing Trick 将原始用户 ID 哈希映射到这 2000 个 token 之一。模型输入由“用户 ID token + 用户历史交互物品对应的语义 ID token 序列”构成。实验发现，加入用户 ID 有助于模型实现个性化召回。

模型的编码器与解码器各包含 4 层 Transformer，每层有 6 个自注意力头，单头维度为 64。各层均采用 ReLU 激活函数，MLP 维度与输入维度分别设为 1024 和 128，dropout 为 0.1。整个模型大约有 1300 万参数。在 Beauty 与 Sports and Outdoors 数据集上，模型训练 200k 步；由于 Toys and Games 数据集更小，仅训练 100k 步。batch size 为 256，学习率在前 10k 步固定为 0.01，之后采用反平方根衰减策略。

4.1 序列推荐性能

本节中，作者将 TIGER 与以下序列推荐方法进行比较：GRU4Rec、Caser、HGN、SASRec、BERT4Rec、FDSA、S³-Rec 以及 P5。需要指出的是，除 P5 外，这些基线模型本质上都在高维向量空间中进行双编码：用户历史交互与候选物品都被映射为高维表示，然后通过最大内积搜索检索用户下一步可能交互的物品。与此不同，TIGER 使用序列到序列模型逐 token 直接预测物品的语义 ID。

作者对 TIGER 在序列推荐任务上的表现进行了广泛分析，并与上述基线进行比较。除 P5 外，其余基线结果来自 Zhou 等人公开发布的结果；P5 则使用作者开源代码进行复现实验。为保证公平比较，本文对 P5 的数据预处理方式做了修正，使其与其他基线及本文方法保持一致，相关细节见附录 D。

实验结果如表 1 所示。可以看到，TIGER 在所有基准数据集上都稳定优于现有基线。在 Beauty 数据集上，TIGER 相比第二优模型有着十分明显的提升：相较 SASRec，NDCG@5 最高提升约 29%；相较 S³-Rec，Recall@5 提升约 17.3%。在 Toys and Games 数据集上，TIGER 在 NDCG@5 与 NDCG@10 上也分别提升约 21% 与 15%。

表 1: 序列推荐性能比较。最后一行为 TIGER 相对最佳基线的提升百分比。

方法	Sports and Outdoors				Beauty				Toys and Games			
	R@5	N@5	R@10	N@10	R@5	N@5	R@10	N@10	R@5	N@5	R@10	N@10
P5	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
Caser	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec	0.0233	0.0154	0.0350	0.0192	0.0387	0.0249	0.0605	0.0318	0.0463	0.0306	0.0675	0.0374
S ³ -Rec	0.0251	0.0161	0.0385	0.0204	0.0387	0.0244	0.0647	0.0327	0.0443	0.0294	0.0700	0.0376
TIGER (本文)	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432
提升	+5.22%	+12.55%	+3.90%	+10.29%	+17.31%	+29.04%	+0.15%	+17.43%	+12.53%	+21.24%	+1.71%	+14.97%

4.2 物品表示分析

本节分析 RQ-VAE 语义 ID 的若干重要性质。首先，作者进行定性分析，以观察语义 ID 的层次化特征；其次，通过与哈希式量化方法比较，评估采用 RQ-VAE 进行量化这一设计选择的重要性；最后，再通过消融实验，比较语义 ID 与随机 ID，考察“使用语义 ID”这一设定本身的价值。

定性分析。 作者在 Amazon Beauty 数据集上分析学习得到的 RQ-VAE 语义 ID。为便于展示，实验中将 RQ-VAE 设为 3 层，其码本大小分别为 4、16 和 256。也就是说，对于某个物品的语义 ID (c_1, c_2, c_3) ，有 $0 \leq c_1 \leq 3$ 、 $0 \leq c_2 \leq 15$ 、 $0 \leq c_3 \leq 255$ 。在图中，作者将每个物品的真实类别按第一层 token c_1 标注，从整体类别分布角度观察不同 c_1 对应的品类结构。实验发现， c_1 能够较好地捕捉物品的高层语义类别，例如某些取值主要对应“Hair”相关产品，而另一些取值则更多对应面部、唇部、眼部等“Makeup”和“Skin”产品。进一步地，当固定 c_1 后，再观察所有可能的 c_2 的类别分布，可以发现第二层码字 c_2 会把由 c_1 表示的粗粒度语义继续细分为更细粒度的类别。也就是说，RQ-VAE 学到的语义 ID 具有明显的层次性，而这正是后续多种新能力的基础。

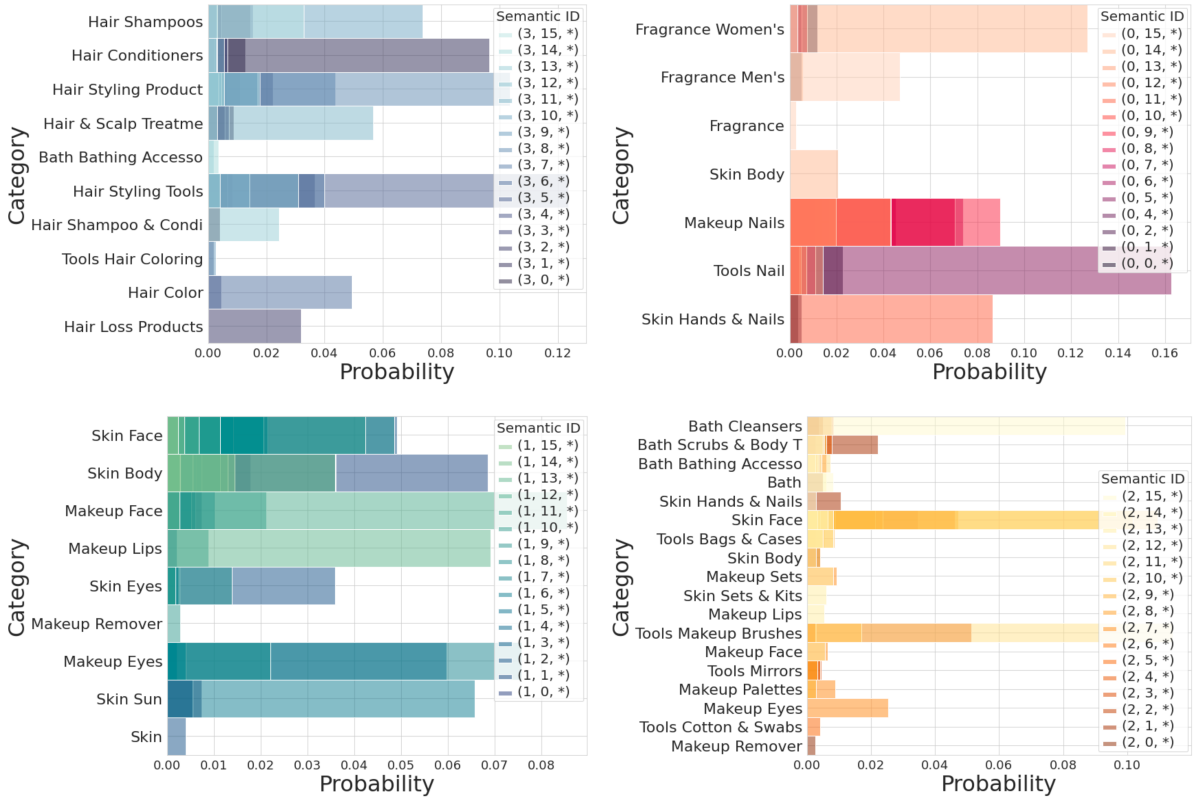


图 3: Amazon Beauty 数据集上的 RQ-VAE 语义 ID 定性分析：第一层 token 更偏向粗粒度类别，第二层与第三层 token 则进一步对应更细粒度语义。

与传统推荐系统基于随机原子 ID 学习物品嵌入不同，TIGER 所使用的语义 ID 使得语义相近的物品能够共享部分码字，从而便于模型在相似物品之间共享知识。

哈希语义 ID 与 RQ-VAE 语义 ID 的比较。 作者通过将 RQ-VAE 与局部敏感哈希 (LSH) 比较，研究 RQ-VAE 在框架中的重要性。LSH 是一种流行的哈希技术，也可以较容易地改造到本文设定中。对于 LSH 语义 ID，作者使用 h 个随机超平面对嵌入向量 x 做随机投影，并将投影结果编码为二进制向量，再将该向量转换为整数码字；重复该过程 m 次，就得到长度为 m 的语义 ID。

表 2 比较了 LSH 语义 ID 与 RQ-VAE 语义 ID 的表现。在该实验中，LSH 设定为 $h = 8$ 且 $m = 4$ ，以使其表示容量与 RQ-VAE 大致可比。随机超平面参数从标准正态分布采样，

从而保证方向具有球对称性。结果表明，RQ-VAE 在三个数据集上都稳定优于 LSH，这说明：在相同的内容语义嵌入基础上，借助非线性深度神经网络学习到的量化方式，比基于随机投影的量化方式更有效。

随机 ID 与语义 ID 的比较。 作者还比较了随机生成 ID 与语义 ID 在生成式检索推荐系统中的差异。对于随机 ID 基线，为每个物品独立随机分配长度为 m 的码字序列，其中每个码字都从 $\{1, 2, \dots, K\}$ 中均匀采样。为了使表示容量与 RQ-VAE 相近，实验中设置 $m = 4$ 、 $K = 255$ 。结果表明，不论是 RQ-VAE 还是 LSH 生成的语义 ID，都显著优于随机 ID。这说明，将基于内容的语义信息引入物品表示，对于生成式推荐是非常重要的。

表 2: 不同 ID 生成技术的消融实验：RQ-VAE 语义 ID 明显优于哈希语义 ID 和随机 ID。

方法	Sports and Outdoors				Beauty				Toys and Games			
	R@5	N@5	R@10	N@10	R@5	N@5	R@10	N@10	R@5	N@5	R@10	N@10
随机 ID	0.0070	0.0050	0.0116	0.0063	0.0296	0.0205	0.0434	0.0250	0.0362	0.0270	0.0448	0.0298
LSH 语义 ID	0.0215	0.0146	0.0321	0.0180	0.0379	0.0259	0.0533	0.0309	0.0412	0.0299	0.0566	0.0349
RQ-VAE 语义 ID	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432

4.3 新能力

作者指出，本文提出的生成式检索框架带来了两类现有序列推荐模型难以直接满足的实际能力：**冷启动推荐**与**推荐多样性**。这两种能力都来源于 RQ-VAE 语义 ID 与生成式检索框架之间的协同作用。

冷启动推荐。 在真实推荐系统中，新物品会不断加入推荐语料库。由于这些新物品在训练数据中缺乏用户曝光与交互记录，依赖随机原子 ID 的传统推荐模型往往很难把它们召回出来。而 TIGER 在预测下一物品时利用了物品语义，因此更容易处理冷启动推荐。

在这一实验中，作者以 Beauty 数据集为例，从训练集中移除 5% 的测试物品，以模拟“新加入但训练时未见”的物品，并将其称为 unseen items。这样的处理保证了对于这些 unseen items 不存在数据泄漏。物品仍采用长度为 4 的语义 ID 表示，其中前三个 token 由 RQ-VAE 生成，第 4 个 token 用于保证 seen items 的唯一性。RQ-VAE 与序列到序列模型都只在训练集上训练；训练完成后，再利用 RQ-VAE 为整个数据集中的所有物品（包括 unseen items）生成语义 ID。

对于模型预测出的语义 ID (c_1, c_2, c_3, c_4) ，系统首先检索训练集中具有完全相同 ID 的 seen item。由于唯一性设计，每个预测的完整语义 ID 至多对应一个训练集物品。此外，所有与该预测在前三个语义 token 上相同的 unseen items，也会被加入候选列表。最终，在生成 Top- K 候选时，作者引入一个超参数 ϵ ，用于控制最终结果中 unseen items 所占比例的上限。

作者将 TIGER 与 kNN 方法在冷启动场景中进行比较。kNN 基线在语义表示空间中进行最近邻搜索，被记作 Semantic_KNN。实验图表显示，当 $\epsilon = 0.1$ 时，TIGER 在所有

Recall@K 指标上都优于 Semantic_KNN；而当 $\epsilon \geq 0.1$ 时，本文方法在 Recall@10 上也始终超过基线。这说明，借助语义 ID 和生成式解码，TIGER 在冷启动推荐方面具有明显优势。

推荐多样性。 除了 Recall 和 NDCG 这些核心指标外，推荐结果的多样性同样是实际系统中极其重要的目标。如果一个推荐系统缺乏多样性，可能会损害用户的长期参与度。作者指出，其生成式检索框架可以通过解码阶段的温度采样，来有效控制推荐结果的多样性。

虽然温度采样从原则上也可以用于其他推荐模型，但 TIGER 的特殊之处在于：由于 RQ-VAE 语义 ID 是层次化的，模型可以在不同层级上进行采样。例如，对语义 ID 的第一个 token 进行采样，相当于在粗粒度类别层面进行探索；而对第二个或第三个 token 进行采样，则更像是在同一类别内部进行细粒度探索。

作者使用 Entropy@K 指标定量衡量多样性，其中熵是根据模型预测 Top-K 物品的真实类别分布计算得到的。实验结果表明，提高解码温度可以显著提高预测结果在类别层面的熵值，也就是提升推荐多样性。作者还给出了若干定性例子，说明随着温度升高，模型给出的前 10 个候选会从单一类别扩展为若干相关类别的组合。

表 3: Beauty 数据集上，不同温度下模型预测类别分布的熵。熵越高，说明推荐结果越多样。

温度	Entropy@10	Entropy@20	Entropy@50
$T = 1.0$	0.76	1.14	1.70
$T = 1.5$	1.14	1.52	2.06
$T = 2.0$	1.38	1.76	2.28

表 4: 基于温度解码的推荐多样性示例。

目标类别	$T = 1.0$ 下 Top-10 最常见类别	$T = 2.0$ 下 Top-10 最常见类别
Hair Styling Products	Hair Styling Products	Hair Styling Products, Hair Styling Tools, Skin Face
Tools Nail	Tools Nail	Tools Nail, Makeup Nails
Makeup Nails	Makeup Nails	Makeup Nails, Skin Hands & Nails, Tools Nail
Skin Eyes	Skin Eyes	Hair Relaxers, Skin Face, Hair Styling Products, Skin Eyes
Makeup Face Tools	Makeup Brushes, Makeup Face Tools	Makeup Brushes, Makeup Face, Skin Face, Makeup Sets, Hair Styling Tools
Hair Loss Products	Hair Loss Products, Skin Face, Skin Body	Skin Face, Hair Loss Products, Hair Shampoos, Hair & Scalp Treatments, Hair Conditioners

4.4 消融实验

作者进一步考察了序列到序列模型层数变化所带来的影响。结果如表中所示：随着网络层数增大，指标略有提升，但提升幅度并不大。作者还测试了是否输入用户信息对结果的影响，相关结果放在附录中。

表 5: 不同 Transformer 层数的效果。

层数	Recall@5	NDCG@5	Recall@10	NDCG@10
3	0.04499	0.03062	0.06699	0.03768
4	0.04540	0.03210	0.06480	0.03840
5	0.04633	0.03206	0.06596	0.03834

4.5 无效 ID

由于模型是以自回归方式逐个解码目标语义 ID 的码字，因此存在生成无效 ID 的可能性，也就是生成出来的 ID 并不对应推荐数据集中的任何物品。在本文实验中，作者使用长度为 4、每个码字基数为 256 的语义 ID，因此理论上的完整 ID 空间大小为 256^4 ，约等于 4 万亿；而实际数据集中的物品数只有 1 万到 2 万级别。因此，从整个 ID 空间来看，真正有效的 ID 只占极小比例。即便如此，作者发现 TIGER 几乎总能生成有效 ID。

实验将“Top- K 预测中无效 ID 的比例”作为指标进行可视化。对于 Top-10 预测，三个数据集上的无效 ID 比例大约只在 0.1% 到 1.6% 之间。若希望始终得到 Top-10 个有效 ID，也可以通过增大 beam size，并过滤掉无效 ID 的方式实现。

作者还指出，即使存在少量无效 ID，TIGER 依然已经优于多种流行的序列推荐方法。进一步的扩展方向之一是：当模型生成无效 token 前缀时，可采用“前缀匹配”机制，检索具有相同语义前缀的物品。考虑到 RQ-VAE token 的层次性，前缀匹配可以理解为模型先预测出一个类别层级，再在该层级下检索物品。这种扩展有可能进一步提升 Recall 和 NDCG，作者将其留作未来工作。

5 结论

本文提出了一种新的推荐召回范式——TIGER，它使用生成模型来完成推荐系统中的候选检索。支撑这一方法的是一种新的物品语义 ID 表示：首先对内容嵌入施加层次量化器 (RQ-VAE)，再由此生成构成语义 ID 的 token。该框架最终得到的是一种无需显式建立传统 ANN 索引即可训练和服务的模型，因为 Transformer 的参数记忆本身就充当了语义索引。

作者指出，与那些需要在训练时为每个物品创建大规模嵌入表、或者在服务时为全部物品建立索引的系统相比，本文方法的嵌入表规模不会随着物品空间大小线性增长，这一点对系统内存开销十分有利。通过在三个数据集上的实验，作者展示了该模型不仅能取得最先进的召回性能，还具备泛化到全新未见物品的能力。

A 附录 A：相关工作的补充

生成式文档检索

传统文档检索通常训练一个双塔模型，将查询与文档映射到同一高维向量空间，再通过 ANN 或 MIPS 搜索相关文档。这类方法的一个问题是往往需要维护庞大的嵌入表。生成式检索则是近年来提出的一种新范式，其思路是逐 token 生成文档标题、文档名称，或者文档 ID 字符串。Cao 等人提出的 GENRE 将实体名逐 token 生成出来；Tay 等人提出 DSI，是首个为每个文档分配结构化语义 DocID，并让模型在给定查询后自回归生成 DocID 的工作。DSI 被视为信息检索领域从匹配式检索向生成式检索转变的重要标志。随后，Lee 等人表明，在多跳检索设定中，生成式文档检索同样有效；Wang 等人则针对语义 DocID 的前缀结构设计了专门解码器；CGR 则尝试结合双编码器与生成式检索各自优势。据作者所述，本文是首个在检索模型中使用由自编码器（RQ-VAE）生成的语义 ID 的工作。

向量量化

向量量化可以理解为：将一个高维向量压缩成一个低维码元组。最直观的一类技术是层次聚类，例如 DSI 中使用的方式：在某一轮形成的簇会在下一轮进一步被划分为子簇。另一类流行方法是 VQ-VAE，它先通过编码器压缩输入，再对压缩后的表示分块量化，最终形成一个码字序列，并由解码器重建原始输入。RQ-VAE 在此基础上对编码器输出进行残差量化，以获得更低的重建误差。LSH 也是常见的聚类与近似最近邻方法，其中本文所用的是 SimHash，它通过随机超平面生成二进制向量，再将其作为哈希码。由于其计算复杂度较低且可扩展性较强，作者将其作为向量量化的一个基线方案。

B 附录 B：基线方法简介

下面简要说明与 TIGER 对比的各个基线：

- **GRU4Rec**：最早的基于 RNN 的序列推荐方法之一，采用定制化 GRU 来建模序列。
- **Caser**：采用 CNN 结构，通过水平与垂直卷积捕捉高阶马尔可夫链特征。
- **HGN**：使用层次门控网络，同时捕捉长期兴趣和短期兴趣。
- **SASRec**：使用带因果掩码的 Transformer 建模用户序列交互。
- **BERT4Rec**：利用双向自注意力 Transformer 解决单向结构的局限。
- **FDSA**：在 Transformer 输入中引入物品特征，而不仅仅是物品嵌入。
- **S³-Rec**：通过自监督预训练提升序列推荐性能。
- **P5**：利用预训练大语言模型，将多种推荐任务统一到单一框架中。

C 附录 C: 数据集统计

作者使用 Amazon Product Reviews 数据集中的三个公开基准。先将用户评论历史按时间戳排序形成物品序列，并过滤掉评论数少于 5 的用户。按照标准评测协议，对每个用户序列采用 leave-one-out 划分：最后一个物品用于测试，倒数第二个物品用于验证，其余物品用于训练。训练时，用户历史长度被截断到 20。

表 6: 三个真实基准数据集的统计信息。

数据集	用户数	物品数	平均序列长度	中位序列长度
Beauty	22,363	12,101	8.87	6
Sports and Outdoors	35,598	18,357	8.32	6
Toys and Games	19,412	11,924	8.63	6

D 附录 D: 对 P5 数据预处理的修改

P5 原始代码在预处理 Amazon 数据集时，会先为每个用户创建按时间顺序排列的会话，然后再将原始物品 ID 重新映射为连续递增的整数 1、2、3、...。也就是说，第一个会话中的第一个物品会被映射为 1，若第二个物品此前未出现，则映射为 2，依此类推。值得注意的是，这一预处理发生在训练集与测试集划分之前。这会导致许多序列呈现出 $a, a+1, a+2, \dots$ 这样的形式。由于 P5 使用 SentencePiece 分词器，训练集和测试集中的物品 ID 子词可能发生共享，从而在推理过程中引入信息泄漏。

为解决这一问题，作者没有使用连续递增整数 ID，而是为物品分配随机整数 ID，随后再进行训练与评估划分。除这一点外，其余 P5 代码保持不变。作者还从零实现了一个只针对序列推荐任务的 P5 版本，结果表明：若采用连续整数序列作为 item ID，确实会导致指标异常偏高或至少不低于论文报告值，从侧面说明原始预处理存在泄漏风险。

表 7: P5 在标准预处理与修正预处理下的结果对比。

方法	Sports and Outdoors				Beauty				Toys and Games			
	R@5	N@5	R@10	N@10	R@5	N@5	R@10	N@10	R@5	N@5	R@10	N@10
P5	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
P5-ours	0.0107	0.0076	0.01458	0.0088	0.0350	0.0250	0.0480	0.0298	0.0180	0.0130	0.0235	0.0150

E 附录 E: 进一步讨论

输入用户信息的影响

作者比较了是否向模型提供用户信息的差异。结果表明，加入用户 ID 后，各项指标均有小幅提升，这说明即便在生成式检索框架下，显式用户身份信息依然有助于个性化建模。

表 8: 向推荐系统提供用户信息的影响。

设定	Recall@5	NDCG@5	Recall@10	NDCG@10
不使用用户信息	0.04458	0.0302	0.06479	0.0367
使用用户 ID (论文主结果)	0.0454	0.0321	0.0648	0.0384

不同随机种子下的均值与标准误

作者报告了在 3 个不同随机种子下, 三组数据集的均值与标准误。可以看到, TIGER 的指标波动非常小, 说明实验结果具有较好的稳定性。

表 9: 不同数据集上指标的均值与标准误 (3 次不同随机种子运行)。

数据集	Recall@5	NDCG@5	Recall@10	NDCG@10
Beauty	0.0441 $\pm$ 0.00069	0.0309 $\pm$ 0.00062	0.0642 $\pm$ 0.00092	0.0374 $\pm$ 0.00061
Sports and Outdoors	0.0278 $\pm$ 0.00069	0.0189 $\pm$ 0.00043	0.0419 $\pm$ 0.0010	0.0234 $\pm$ 0.00048
Toys and Games	0.0518 $\pm$ 0.00064	0.0375 $\pm$ 0.00039	0.0698 $\pm$ 0.0013	0.0433 $\pm$ 0.00047

语义 ID 长度与码本大小的影响

作者尝试改变语义 ID 的长度和码本大小, 例如把 ID 改为由 6 个码字组成、每个码字来自大小为 64 的码本。实验发现, TIGER 的推荐指标对这些变化总体较为鲁棒。不过, 语义 ID 越长, 输入序列就越长, 因此基于 Transformer 的序列到序列模型在计算上会更昂贵。

可扩展性

为测试语义 ID 的可扩展性, 作者将三个数据集合并, 在整个合并物品集上统一生成语义 ID, 再将这些语义 ID 用于 Beauty 数据集上的推荐任务。随后把该结果与“仅在 Beauty 物品集上生成语义 ID”进行对比。实验表明, 性能只有小幅下降, 说明语义 ID 在跨数据集合并场景下具有一定可扩展性。

表 10: 在合并数据集上生成语义 ID 与仅在 Beauty 数据集上生成语义 ID 的对比。

设定	Recall@5	NDCG@5	Recall@10	NDCG@10
Semantic ID (合并数据集)	0.04355	0.3047	0.06314	0.03676
Semantic ID (仅 Beauty)	0.0454	0.0321	0.0648	0.0384

推理成本

尽管 TIGER 在序列推荐任务上表现出色, 但作者也坦率指出: 由于需要通过 beam search 执行自回归解码, 因此它在推理阶段可能比基于 ANN 的模型更耗计算资源。作者强

调, 本文工作的核心目标并非优化推理效率, 而是开启“基于生成式检索的推荐系统”这一新研究方向。未来工作将考虑压缩模型规模, 或者探索其他提升推理效率的方式。

查找表与嵌入表的内存成本

TIGER 维护两个哈希查找表: 其一是 “Item ID $\rightarrow$ Semantic ID”, 其二是 “Semantic ID $\rightarrow$ Item ID”。这两个表都只在 RQ-VAE 训练完成后生成一次, 随后在序列到序列模型训练中保持冻结。每个语义 ID 由 4 个整数构成, 每个整数占 8 bit, 因此每个物品的语义 ID 共占 32 bit; 而物品原始 ID 也可存为 32 bit 整数。于是, 每个查找表的规模大约是 $64N$ bit, 其中 N 为物品数。

与传统推荐系统相比, TIGER 的嵌入表显著更小。传统推荐系统通常为每个物品维护一个嵌入, 因此需要 $N \times d$ 规模的嵌入表; 而 TIGER 只需为每个语义码字维护嵌入。本文实验中, 共使用 4 个位置、每个位置 256 个码字, 因此只需维护 1024 个码字嵌入, 也就是内存规模约为 $1024d$ 。当 N 处于 1 万到 2 万量级时, 这种差异就已经十分明显; 如果物品规模进一步扩大, TIGER 在嵌入内存上的优势还会更加突出。

参考文献 (原文保留)

由于参考文献条目主要承担检索与溯源功能, 以下保留原文英文书目信息, 不再逐条翻译:

1. Himan Abdollahpour, Masoud Mansoury, Robin Burke, and Bamshad Mobasher. The unfairness of popularity bias in recommendation. arXiv preprint arXiv:1907.13286, 2019.
2. Moses S Charikar. Similarity estimation techniques from rounding algorithms. In Proceedings of the thirty-fourth annual ACM symposium on Theory of computing, pages 380–388, 2002.
3. Heng-Tze Cheng et al. Wide & deep learning for recommender systems. In Proceedings of the 1st workshop on deep learning for recommender systems, pages 7–10, 2016.
4. Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for YouTube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198, 2016.
5. Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. arXiv preprint arXiv:2010.00904, 2020.
6. Gabriel de Souza Pereira Moreira et al. Transformers4rec: Bridging the gap between NLP and sequential/session-based recommendation. In RecSys, pages 143–153, 2021.
7. Jacob Devlin et al. BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805, 2018.
8. Shijie Geng et al. Recommendation as language processing (P5). arXiv preprint arXiv:2203.13366, 2022.

9. Carlos A Gomez-Uribe and Neil Hunt. The Netflix recommender system. *ACM TMIS*, 6(4):1–19, 2015.
10. Ruining He and Julian McAuley. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *WWW*, pages 507–517, 2016.
11. Balázs Hidasi et al. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939*, 2015.
12. Yupeng Hou et al. Learning vector-quantized item representation for transferable sequential recommenders. *arXiv preprint arXiv:2210.12316*, 2022.
13. Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: toward removing the curse of dimensionality. In *STOC*, pages 604–613, 1998.
14. Piotr Indyk et al. Locality-preserving hashing in multidimensional spaces. In *STOC*, pages 618–625, 1997.
15. Herve Jegou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE TPAMI*, 33(1):117–128, 2010.
16. Wang-Cheng Kang et al. Learning to embed categorical features without embedding tables for recommendation. *arXiv preprint arXiv:2010.10784*, 2020.
17. Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *ICDM*, pages 197–206, 2018.
18. Dongmoon Kim et al. A music recommendation system with a dynamic k-means clustering algorithm. In *ICMLA*, pages 399–403, 2007.
19. Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
20. Tim Kraska et al. The case for learned index structures. In *SIGMOD*, pages 489–504, 2018.
21. Doyup Lee et al. Autoregressive image generation using residual quantization. In *CVPR*, pages 11523–11532, 2022.
22. Hyunji Lee et al. Contextualized generative retrieval. *arXiv preprint arXiv:2210.02068*, 2022.
23. Hyunji Lee et al. Generative retrieval for long sequences. *arXiv preprint arXiv:2204.13596*, 2022.
24. Jing Li et al. Neural attentive session-based recommendation. In *CIKM*, pages 1419–1428, 2017.
25. Chen Ma, Peng Kang, and Xue Liu. Hierarchical gating networks for sequential recommendation. In *KDD*, pages 825–833, 2019.
26. Masoud Mansoury et al. Feedback loop and bias amplification in recommender systems, 2020.

27. Jianmo Ni et al. Sentence-T5: Scalable sentence encoders from pre-trained text-to-text models. In Findings of ACL 2022, pages 1864–1874, 2022.
28. Adam Roberts et al. Scaling up models and data with T5X and SeqIO. arXiv preprint arXiv:2203.17189, 2022.
29. Rico Sennrich et al. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909, 2015 / ACL 2016.
30. Anima Singh et al. Better generalization with semantic IDs: A case study in ranking for recommendations. arXiv preprint arXiv:2306.08121, 2023.
31. Fei Sun et al. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In CIKM, pages 1441–1450, 2019.
32. Jiaxi Tang and Ke Wang. Personalized top-N sequential recommendation via convolutional sequence embedding. In WSDM, pages 565–573, 2018.
33. Yi Tay et al. Transformer memory as a differentiable search index. arXiv preprint arXiv:2202.06991, 2022.
34. Aaron van den Oord, Oriol Vinyals, et al. Neural discrete representation learning. NeurIPS, 30, 2017.
35. Ashish Vaswani et al. Attention is all you need. NeurIPS, 30, 2017.
36. Yujing Wang et al. A neural corpus indexer for document retrieval. arXiv preprint arXiv:2206.02743, 2022.
37. Kilian Weinberger et al. Feature hashing for large scale multitask learning. In ICML, pages 1113–1120, 2009.
38. Xinyang Yi et al. Sampling-bias-corrected neural modeling for large corpus item recommendations. In RecSys, pages 269–277, 2019.
39. Neil Zeghidour et al. SoundStream: An end-to-end neural audio codec. CoRR, abs/2107.03312, 2021.
40. Shuai Zhang et al. Next item recommendation with self-attention. arXiv preprint arXiv:1808.06414, 2018.
41. Tingting Zhang et al. Feature-level deeper self-attention network for sequential recommendation. In IJCAI, pages 4320–4326, 2019.
42. Zhe Zhao et al. Recommending what video to watch next: a multitask ranking system. In RecSys, pages 43–51, 2019.
43. Kun Zhou et al. S3-Rec: Self-supervised learning for sequential recommendation with mutual information maximization. In CIKM, pages 1893–1902, 2020.